- *Configuration service*—This provides configuration data persistence, auditing, and accountability. It's useful for defining configuration data specific to the application, such as feature flags, thread pools, connection pools, timeouts, and URLs for third-party services. We adopted this strategy with Spring Cloud Config.

Those three strategies are generic enough that we can use them to configure applications for any cloud environment and service model (CaaS, PaaS, FaaS). When it comes to Kubernetes, there's an additional configuration strategy that is provided natively by the platform: ConfigMaps and Secrets.

These are a very convenient way to define configuration data that depends on the infrastructure and platform where the application is deployed: service names (defined by Kubernetes `Service` objects), credentials and certificates for accessing other services running on the platform, graceful shutdown, logging, and monitoring. You could use ConfigMaps and Secrets to complement or completely replace what a configuration service does. Which you choose depends on the context. In any case, Spring Boot provides native support for all those options.

For the Polar Bookshop system, we'll use ConfigMaps and Secrets instead of the Config Service to configure applications in Kubernetes environments. Still, all the work we've done so far on Config Service would make including it in the overall deployment of Polar Bookshop on Kubernetes straightforward. In this section, I'll share some final considerations for making Config Service production-ready, in case you'd like to expand on the examples and include it in the final deployment in production.

14.1.1 Securing the configuration server with Spring Security